

進階程式設計

Advanced Program Design

L5



Chia-Chi Chang
ccchang@cc.ncue.edu.tw

四捨五入取至小數點以下第2位

- 使用%.2f用來取至小數點以下第2位，並不保證四捨五入
- 使用setprecision(2)用來取至小數點以下第2位，並不保證四捨五入
- 四捨五入應使用round()[※#include <math.h>]

```
01 #include <stdio.h>
02 #include <stdlib.h>
03 #include <math.h>
04 int main(int argc, char *argv[]) {
05     double a = 3.185, b = 3.195;
06
07     printf("a = %.2f, b = %.2f\n", a, b);
08     printf("=====\n");
09     printf("a = %.2f, b = %.2f\n", round(a*100.0)/100.0, round(b*100.0)/100.0);
10     return 0;
11 }
```

C

```
01 #include <iostream>
02 #include <string>
03 #include <iomanip>
04 using namespace std;
05 int main(int argc, char** argv) {
06     double a = 3.185, b = 3.195;
07
08     cout<<fixed<<"a = "<<setprecision(2)<<a<<"", b = "<<b<<endl;
09     return 0;
10 }
```

C++

```
a = 3.19, b = 3.19
=====
a = 3.19, b = 3.20
-----
Process exited after 0.0121 seconds with return value 0
Press any key to continue . . .
```

```
a = 3.19, b = 3.19
-----
Process exited after 0.01456 seconds with return value 0
Press any key to continue . . .
```

a*100與a*100.0的差別

```
01 #include <stdio.h>
02 #include <stdlib.h>
03
04 int main(int argc, char *argv[]) {
05     float r = 3007.76489;
06     printf("r * 100 = %.5f\n", r * 100);
07     printf("r * 100.0 = %.5f\n", r * 100.0);
08     return 0;
09 }
```

```
r * 100 = 300776.50000
r * 100.0 = 300776.48926
```

```
-----
Process exited after 0.01297 seconds with return value 0
Press any key to continue . . .
```

- 100觸發單精度浮點數運算，其有效位元僅7位，導致捨入誤差
- 100.0則觸發倍精度浮點數運算，可以保留更多準確位數

Single-Precision Floating-Point Number

單精度浮點數

- Sign bit(符號位元) : 1 bit, where 0 represents positive and 1 represents negative.
- Exponent part(指數偏移值) : 8 bits, using the **Excess 127** representation (subtracting 127 from the binary value to obtain the actual stored value).
- Mantissa part(尾數部份) : 23 bits, starting from the normalized decimal point, with trailing zeros added if necessary.

IEEE-754 Floating Point Converter

Translations: [de](#)

This page allows you to convert between the decimal representation of a number (like "1.02") and the binary format used by all modern CPUs (a.k.a. "IEEE 754 floating point").

IEEE 754 Converter, 2024-02

	Sign	Exponent	Mantissa
Value:	+1	2^3	$1 + 0.0625$
Encoded as:	0	130	524288
Binary:	☐	☒ ☐ ☐ ☐ ☐ ☐ ☒ ☐	☐ ☐ ☒ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐
Decimal Representation	8.5		
Value actually stored in float:	8.5		
Error due to conversion:	0		
Binary Representation	01000001000010000000000000000000		
Hexadecimal Representation	41080000		

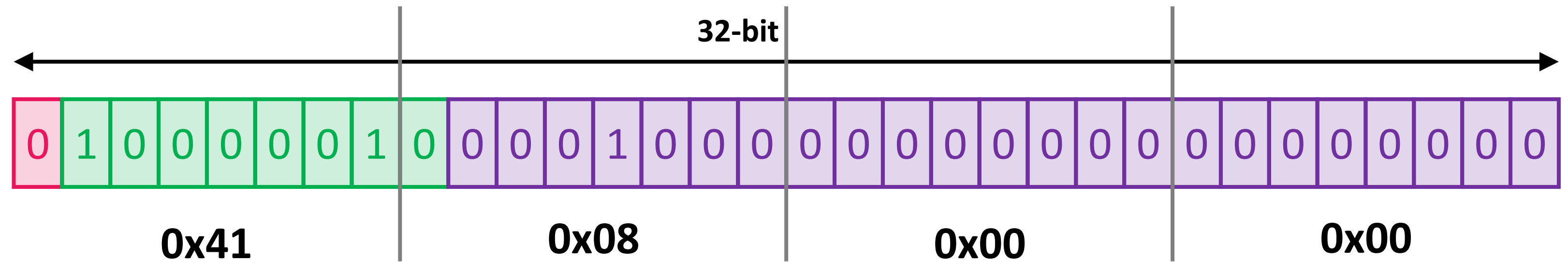
Example : 8.5

STEP 1: $8.5 = (1000.1)_2 = (1.0001 \times 2^3)_2$

STEP 2: Since it is a positive number, the sign bit is 0.

STEP 3: By using the Excess 127 representation, the exponent part is $127+3=130=(10000010)_2$

STEP 4: The mantissa part is 000100000000000000000000

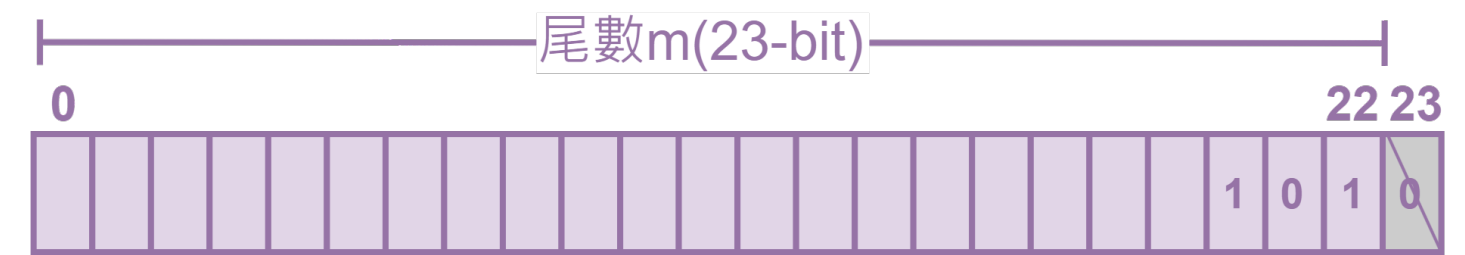
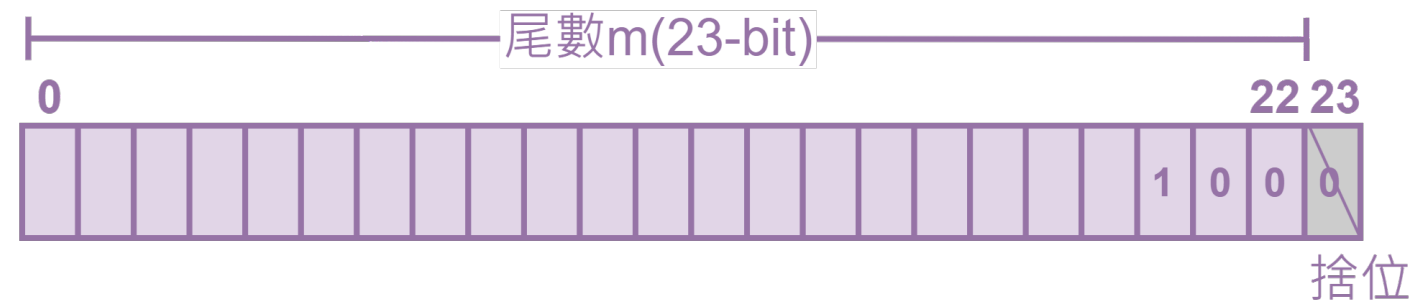


Single Precision
IEEE 754 Floating-Point Standard

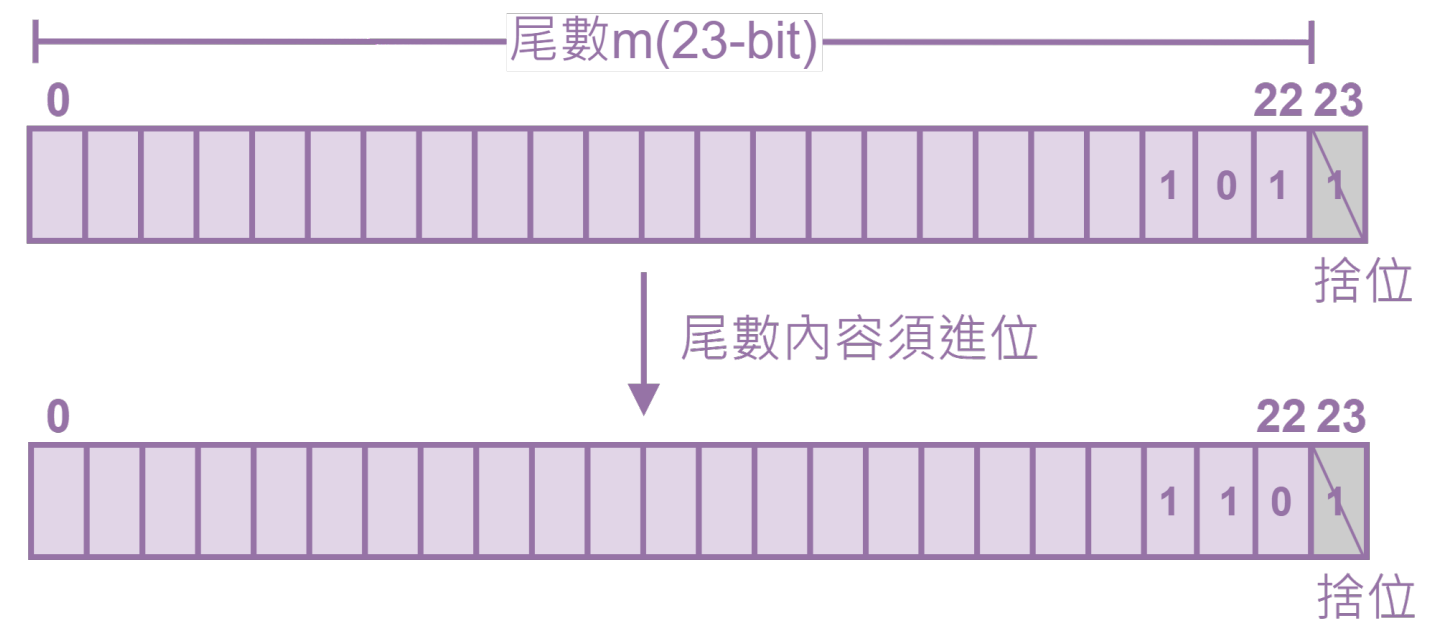
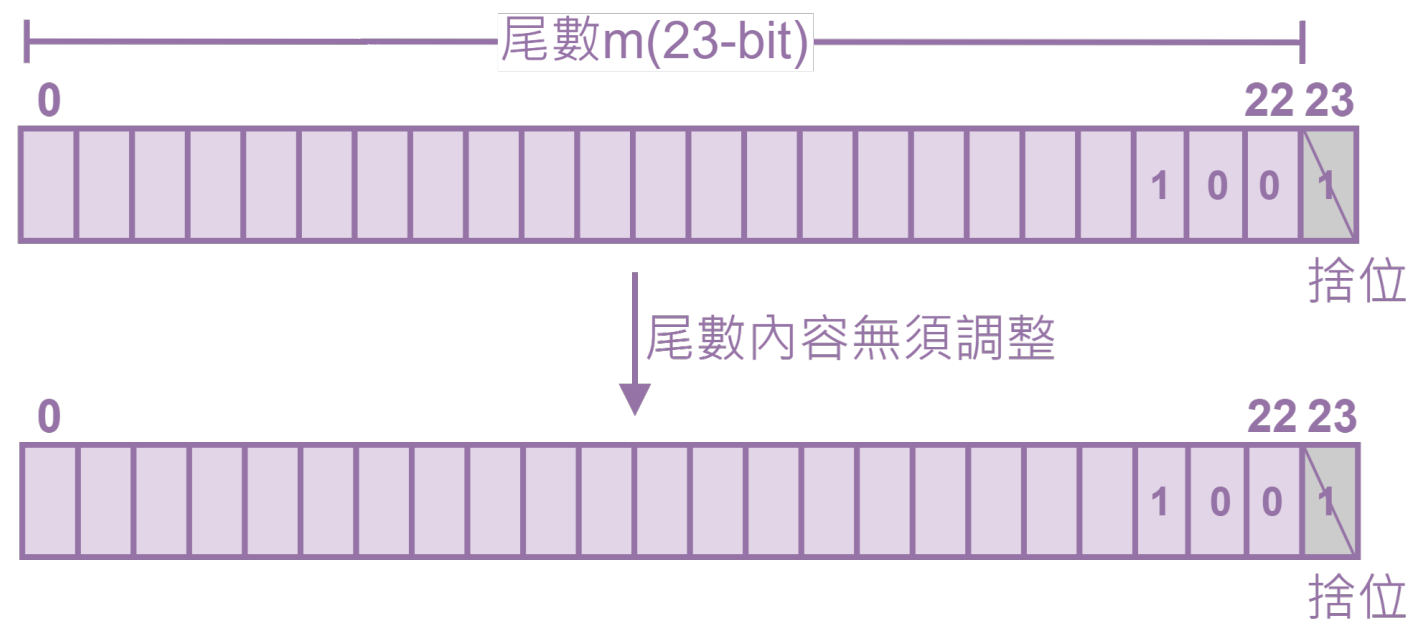
單精度浮點數捨位(truncation)/進位(rounding)規則

➤ 若小數部份正好為24-bit

1. 若第23-bit位元為0：無論第22-bit為何，只取尾數部份(第0位元至第22位元)，第23位元捨位。



2. 若第23-bit位元為1且第22-bit為0時：尾數部份無須進位(第0位元至第22位元內容)，第23位元捨位。
3. 若第23-bit位元為1且第22-bit為1時：尾數部份須進位(第0位元至第22位元內容+1)，第23位元捨位。



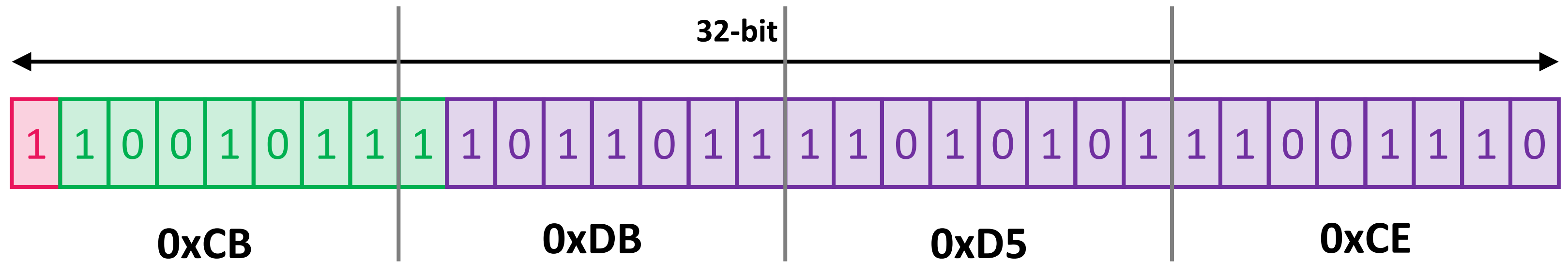
Example 1 – -28814237

STEP 1: $-28814237 = -(1.101101111010101110011101 \times 2^{24})_2$

STEP 2: Since -28814237 is a negative number, the sign bit is 1.

STEP 3: By using the Excess 127 representation, the exponent part is $127 + 24 = 151 = (10010111)_2$

STEP 4: The mantissa part will be 101101111010101110011101
 $(101101111010101110011101)$



Single Precision
IEEE 754 Floating-Point Standard

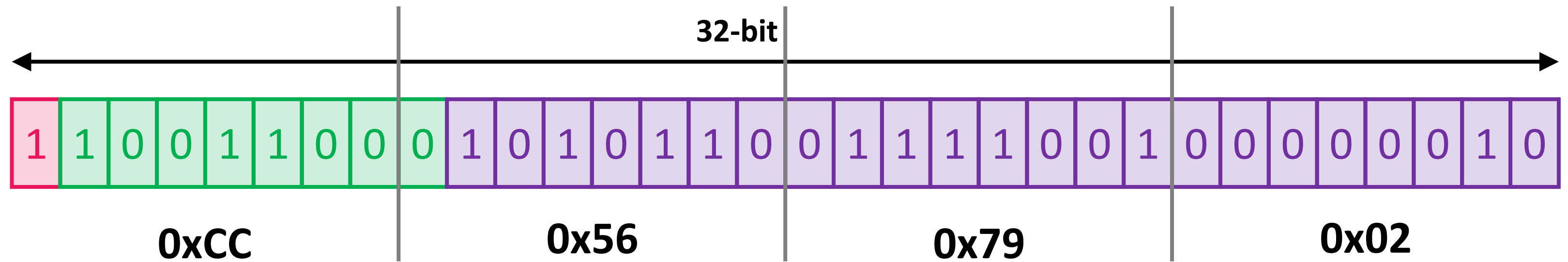
Example 2 – -56222726

STEP 1: $-56222726 = -(1.10101100111100100000000110 \times 2^{25})_2$

STEP 2: Since -56222726 is a negative number, the sign bit is **1**.

STEP 3: By using the Excess 127 representation, the exponent part is $127 + 25 = 152 = (10011000)_2$

STEP 4: The mantissa part will be 10101100111100100000000 **1** 1
(101011001111001000000010 **1**)

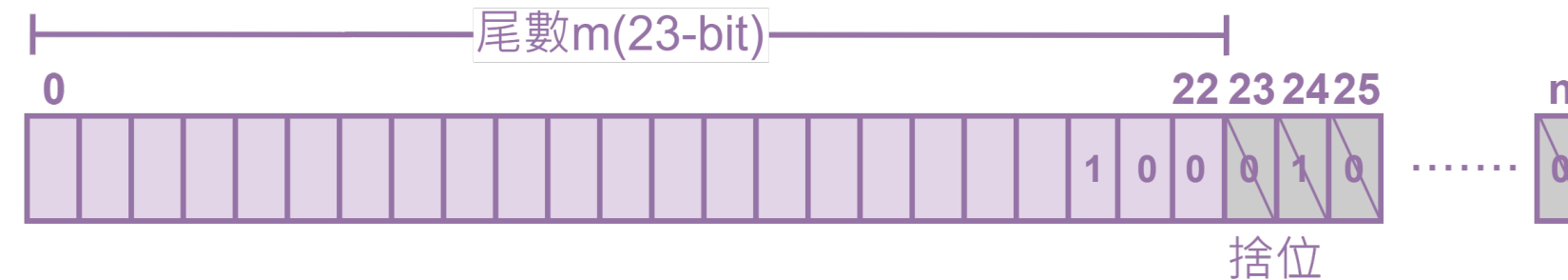


Single Precision
IEEE 754 Floating-Point Standard

單精度浮點數捨位(truncation)/進位(rounding)規則

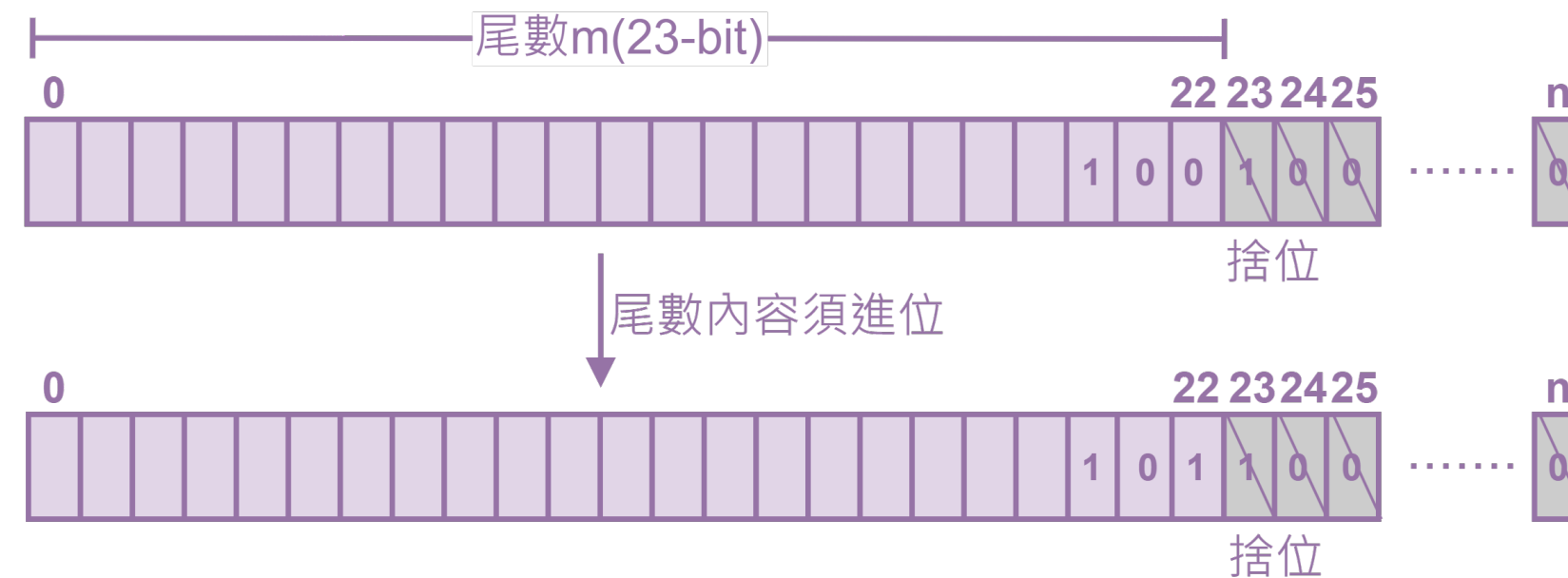
➤ 若小數部份超過24-bit

1. 檢查第23-bit位元為0：取尾數部份(第0位元至第22位元)，第23位元捨位。



2. 檢查第23-bit位元為1：尾數部份**必須進位**(第0位元至第22位元內容+1)，第23位元捨位。

❖ 假設尾數後3位為100，由於第23位元為1，尾數內容須進位成101。



Example 3– -0.1

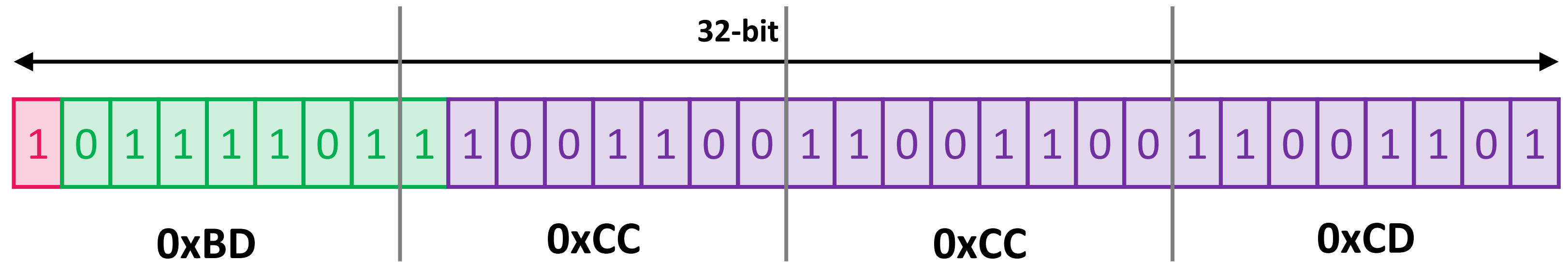
STEP 1: $-0.1 = -(1.10011001100110011001100110011001100110011001100110... \times 2^{-4})_2$

STEP 2: Since -0.1 is a negative number, the sign bit is 1.

STEP 3: By using the Excess 127 representation, the exponent part is $127 - 4 = 123 = (01111011)_2$

STEP 4: The mantissa part will be 100110011001100110011001100110011001100110011001100...

(10011001100110011001101 1100...)



Single Precision
IEEE 754 Floating-Point Standard

Example 4 – -0.751

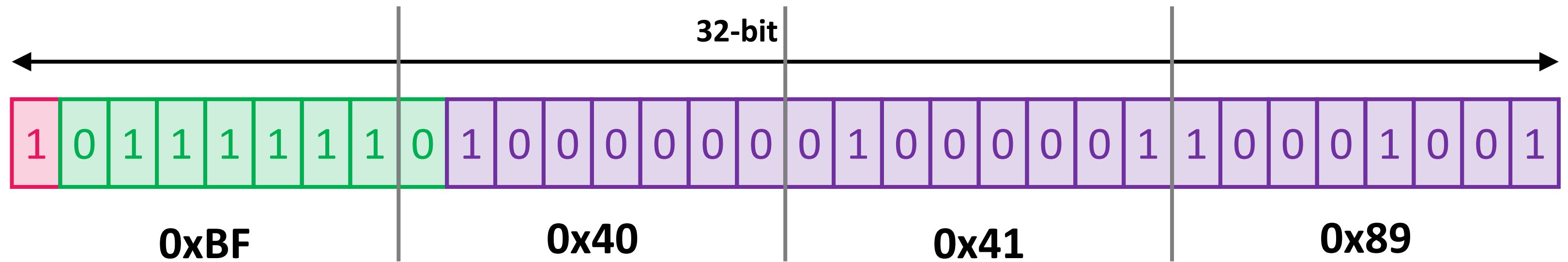
STEP 1: $-0.751 = -(1.100000001000001100010010011011 \times 2^{-1})_2$

STEP 2: Since -0.751 is a negative number, the sign bit is 1.

STEP 3: By using the Excess 127 representation, the exponent part is $127 - 1 = 126 = (01111110)_2$

STEP 4: The mantissa part will be $100000001000001100010010011011...$

$(10000000100000110001001 \text{ } \cancel{0011011...})$



Single Precision
IEEE 754 Floating-Point Standard

使用指標存取浮點數

```
01 #include <stdio.h>
02 #include <stdlib.h>
03 #include <stdint.h>
04 int main(int argc, char *argv[]) {
05     float i = -0.751;
06
07     uint8_t* ptr = &i;
08     for(int32_t loop = 0; loop < 4; loop++) {
09         printf("ptr[%d] = 0x%X\n", loop, *(ptr+loop));
10     }
11     return 0;
12 }
```

```
01 #include <stdio.h>
02 #include <stdlib.h>
03 #include <stdint.h>
04 typedef union {
05     float i;
06     uint32_t num;
07 } uFloat;
08 int main(int argc, char *argv[]) {
09     uFloat s;
10
11     s.i = -0.751;
12     printf("%f\n", s.i);
13     printf("0x%X\n", s.num);
14     printf("num[0] = 0x%X\n", s.num & 0xFF);
15     printf("num[1] = 0x%X\n", (s.num >> 8) & 0xFF);
16     printf("num[2] = 0x%X\n", (s.num >> 16) & 0xFF);
17     printf("num[3] = 0x%X\n", (s.num >> 24) & 0xFF);
18     return 0;
19 }
```

```
ptr[0] = 0x89
ptr[1] = 0x41
ptr[2] = 0x40
ptr[3] = 0xBF
```

```
-----
Process exited after 0.00692 seconds with return value 0
Press any key to continue . . .
```

The End

